Gestion des événements universitaires

Équipe Warriors

Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

Scannez : dépôt Git

Sommaire

01. Problème et périmètre

02. Acteurs et fonctionnalités

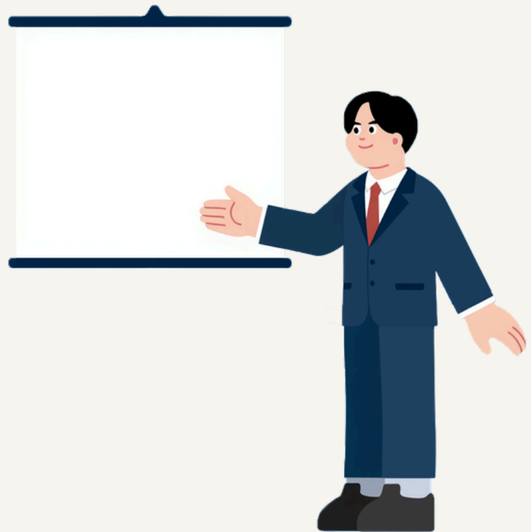
03. Découpage en six microservices

04. Conception métier des microservices

05. Conception du microservice commun

06. Communications synchrones et asynchrones

07. Architecture technique



Problème et périmètre

Le constat

- Inscriptions et listes de présence tenues dans des fichiers séparés
- Salles réservées à la main : risque de double réservation
- Entrée non contrôlée, aucun bilan de fréquentation ni d’avis

Notre réponse

Une plateforme unique pour les conférences, hackathons et soutenances : inscription, réservation de salle, badge QR, contrôle d’entrée, avis et statistiques.

Hors périmètre : paiement en ligne, retransmission vidéo, gestion des comptes utilisateurs (déléguée à Keycloak).

Acteurs et fonctionnalités

Participant

PARTICIPANT

- Consulter les événements publiés
- S’inscrire et recevoir son badge QR
- Donner un avis après l’événement

Organisateur

ORGANISATEUR

- Créer, publier et annuler un événement
- Réserver une salle, affecter les intervenants
- Scanner les badges, suivre les statistiques

Intervenant

INTERVENANT

- Déclarer ses disponibilités
- Consulter ses affectations
- Recevoir sa convocation

Administrateur

ADMIN

- Gérer le parc de salles
- Superviser les événements
- Gérer les modèles de messages

Les comptes et les rôles sont gérés par Keycloak : aucun microservice utilisateur.

Découpage en six microservices

Un contexte métier borné par service, une base de données par service, aucune base partagée.

Événements

Cycle de vie des événements et inscriptions avec contrôle de capacité

Eya Ben Saber — MySQL

Planning des salles

Salles, réservations et détection des conflits de créneau

Amal Barhoumi — PostgreSQL

Check-in / QR

Badges QR, validation d'entrée et enregistrement de la présence

Rayen Amri — MySQL

Feedback et intervenants

Avis des participants, profils et affectations des intervenants

Youssef Saibi — PostgreSQL

Statistiques

Affluence, taux de présence et note moyenne, en lecture seule

Eya Baccouche — PostgreSQL

Notifications

Confirmations, rappels, badges et convocations

Hana Abidi — MongoDB

MICROSERVICE 1 SUR 6 · EYA BEN SABER

Événements

Cycle de vie des événements (brouillon, publié, terminé, annulé) et inscriptions des participants, avec contrôle de capacité.

Données

MySQL

Événement : id, titre,description,type, date, salleId, capacité, statut

Inscription : id, événementId, participantId (Keycloak), statut, date

Pourquoi MySQL : relations événement ↔ inscription et contrôle de capacité transactionnel.

API REST

Méthode	URI	Codes
POST	/events	201 · 400
GET	/events?type=&statut=	200
PUT	/events/{id}/publish	200 · 409
POST	/events/{id}/registrations	201 · 409
DELETE	/registrations/{id}	204 · 404

Interactions

SYNC — Appelle Planning pour réserver la salle ; si Planning est indisponible, l'événement reste en brouillon.

ASYNC — Publie InscriptionConfirmée, ÉvénementPublié, ÉvénementAnnulé et ÉvénementTerminé.

MICROSERVICE 2 SUR 6 · AMAL BARHOUMI

Planning des salles

Gère les salles et leurs réservations, et refuse tout chevauchement de créneau.

Données

PostgreSQL

Salle : id, nom, bâtiment, capacité

Réservation : id, salleId, événementId, début, fin, statut

Pourquoi PostgreSQL : une contrainte d'exclusion sur les plages horaires garantit l'absence de conflit côté base.

API REST

Méthode	URI	Codes
GET	/rooms?minCapacity=	200
POST	/rooms	201 · 403
POST	/reservations	201 · 409 · 422
DELETE	/reservations/{id}	204 · 404
GET	/rooms/{id}/availability	200 · 404

Interactions

SYNC — Appelé par Événements ; n'appelle aucun service, la capacité requise arrive dans la requête.

ASYNC — Consomme ÉvénementAnnulé : la salle réservée est libérée.

MICROSERVICE 3 SUR 6 · RAYEN AMRI

Check-in / QR

Génère un badge QR pour chaque inscription confirmée et valide l'entrée : un badge n'est utilisable qu'une fois.

Données

MySQL

Badge : id, inscriptionId, événementId, participantId, code unique, statut (ACTIF, UTILISÉ, RÉVOQUÉ)

Présence : id, badgeId, horodatage d'entrée

Pourquoi MySQL : l'unicité du code QR est garantie par une contrainte de la base.

API REST

Méthode	URI	Codes
GET	/badges/me	200
POST	/checkins	201 · 404 · 409
GET	/events/{id}/attendance	200
PUT	/badges/{id}/revoke	200 · 404

Interactions

SYNC — Appelle Événements pour vérifier que l'événement est ouvert ; repli sur une copie locale de son état.

ASYNC — Consomme InscriptionConfirmée (crée le badge) ; publie BadgeGénéré et PrésenceEnregistrée.

MICROSERVICE 4 SUR 6 · YOUSSEF SAIBI

Feedback et intervenants

Recueille les avis des participants présents ; gère les profils d'intervenants et de jurys et leurs affectations.

Données

PostgreSQL

Avis : id, événementId, participantId, note (1 à 5), commentaire, date

Intervenant : id, nom, fonction, disponibilités

Affectation : id, intervenantId, événementId, rôle (CONFÉRENCIER, JURY)

Pourquoi H2 : schéma simple et faible volume.

API REST

Méthode	URI	Codes
POST	/events/{id}/feedback	201 · 403 · 409
GET	/events/{id}/feedback	200
POST	/speakers	201 · 400
POST	/assignments	201 · 409
GET	/speakers/{id}/assignments	200 · 404

Interactions

SYNC — Appelle Événements pour valider l'événement lors d'une affectation ; refus si indisponible.

ASYNC — Consomme PrésenceEnregistrée et ÉvénementTerminé ; publie AvisEnvoyé et IntervenantAffecté.

MICROSERVICE 5 SUR 6 · EYA BACCOUCHE

Statistiques

Calcule en lecture seule l’affluence, le taux de présence et la note moyenne d’un événement.

Données

PostgreSQL

StatÉvénement : événementId, inscrits, présents, tauxPrésence, noteMoyenne, nbAvis

Instantané : événementId, date, inscrits, présents

Pourquoi PostgreSQL : agrégations SQL et séries par date.

API REST

Méthode	URI	Codes
GET	/stats/events/{id}	200 · 404
GET	/stats/events?from=&to=	200
GET	/stats/overview	200
GET	/stats/events/{id}/timeline	200 · 404

Interactions

ASYNC — Consomme InscriptionConfirmée, PrésenceEnregistrée, AvisEnvoyé et les événements d’état.

PANNE — Aucun appel vers un autre service : si Statistiques est arrêté, les messages attendent dans le broker.

MICROSERVICE 6 SUR 6 · HANA ABIDI

Notifications

Envoie confirmations, rappels, badges et convocations.
N’appelle aucun service métier : il réagit aux événements.

Données

MongoDB

Notification : id, destinataireId, type, contenu (document libre selon le type), canal, statut, dateEnvoi

Gabarit : type, sujet, corps

Pourquoi MongoDB : contenus variables selon le type de notification, écritures fréquentes.

API REST

Méthode	URI	Codes
GET	/notifications/me	200
PUT	/notifications/{id}/read	200 · 404
GET	/notifications?type=	200 · 403
POST	/templates	201 · 403

Interactions

ASYNC — Consomme InscriptionConfirmée, BadgeGénéré, ÉvénementPublié, ÉvénementAnnulé, ÉvénementTerminé et IntervenantAffecté.

PANNE — Envoi impossible : la notification passe au statut ÉCHEC, sans bloquer les autres services.

SERVICE COMMUN

API Gateway & Infrastructure

Point d'entrée unique, sécurité JWT, découverte des services et configuration centralisée.

Composants

Spring Cloud

API Gateway (8080) : point d'entrée unique du Front-end React.

Eureka Server (8761): annuaire dynamique des 6 microservices.

Config Server (8888): centralisation des fichiers application.yml

Routage Gateway

Préfixe URI	Service cible	Route Eureka
/events/**	Événements	lb://evenements
/rooms/**, /reservations/**	Planning des salles	lb://planning
/badges/**, /checkins/**	Check-in / QR	lb://checkin
/feedback/**, /speakers/**	Feedback	lb://feedback
/stats/**	Statistiques	lb://statistiques
/notifications/**	Notifications	lb://notifications

Interactions

SÉCURITÉ Vérifie le jeton JWT Keycloak avant de router les requêtes du Front-end React (5173)

DOCKER Docker Compose orchestre bases, RabbitMQ, Keycloak et les services

Communication synchrone (OpenFeign)

Trois appels REST entre services, uniquement quand une réponse immédiate est nécessaire.

Événements → Planning

Réserver la salle à la publication

Pourquoi synchrone

La réponse doit être immédiate : un conflit de créneau refuse la publication.

Si l'appelé est arrêté

Publication refusée (503) ; l'événement reste en brouillon.

Check-in → Événements

L'événement est-il ouvert à l'entrée ?

Pourquoi synchrone

La décision doit être prise au moment du scan, devant le participant.

Si l'appelé est arrêté

Repli sur la copie locale de l'état de l'événement.

Feedback → Événements

Valider l'événement lors d'une affectation

Pourquoi synchrone

Une affectation n'a de sens que sur un événement existant, aux dates connues.

Si l'appelé est arrêté

Affectation refusée ; l'organisateur peut réessayer.

Communication asynchrone (événements)

InscriptionConfirmée

Événements → Check-in, Notifications, Statistiques

Pourquoi asynchrone

L'inscription ne doit dépendre ni d'un envoi de message ni d'un calcul de statistique.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

PrésenceEnregistrée

Check-in → Statistiques, Feedback

Pourquoi asynchrone

Alimente l'affluence et ouvre le droit de donner un avis, sans ralentir le scan.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

ÉvénementPublié · Annulé · Terminé

Événements → Notifications, Planning, Feedback, Statistiques

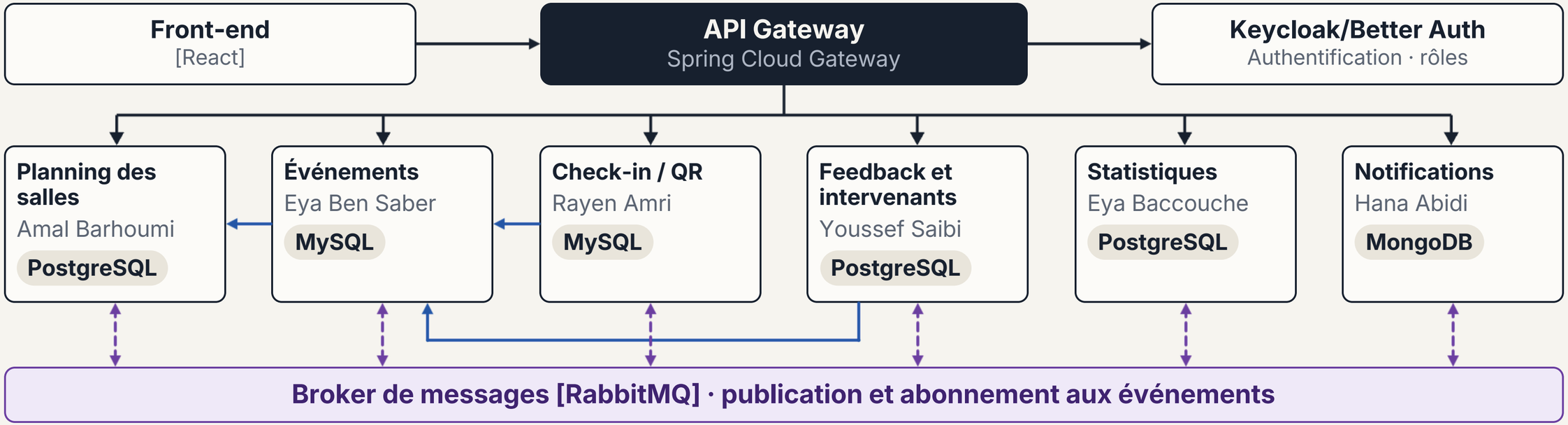
Pourquoi asynchrone

Plusieurs abonnés, et l'émetteur n'a pas à les connaître.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

Architecture technique



INFRASTRUCTURE SPRING CLOUD ET CONTENEURS

Eureka Server
Annuaire des services

Config Server
Configuration centralisée

Docker Compose
Un conteneur par service

Appel REST synchrone (OpenFeign)

Événement asynchrone

Routage de la Gateway

Merci de votre attention

Gestion des événements universitaires · Équipe Warriors

· Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

